# Algorytm Harmony Search

receptura batona proteinowego ALGO-BAR

Każdy wzór rozbity na: co teraz robię i dlaczego

## O co w ogóle chodzi?

Masz baton proteinowy o masie 100 g złożony z czterech składników. Chcesz zrobić go **jak najtaniej**, ale musi mieć **co najmniej 20 g białka** i **nie więcej niż 20 g tłuszczu**.

Brute-force (sprawdź wszystkie kombinacje) jest niemożliwy — kombinacji są miliardy. Harmony Search **uczy się** dobrych receptur i stopniowo je ulepsza.

**Składniki (stałe dane, nie zmieniają się):**

| | Składnik | Białko | Tłuszcz | Cena | Ile może być |
|---|---|---|---|---|---|
| $x_1$ | Izolat białkowy | 90% | 0% | 0,18 PLN/g | od 10 do 40 g |
| $x_2$ | Pasta migdałowa | 25% | 50% | 0,06 PLN/g | od 15 do 40 g |
| $x_3$ | Syrop ryżowy | 0% | 0% | 0,025 PLN/g | od 15 do 35 g |
| $x_4$ | Quinoa | 14% | 6% | 0,055 PLN/g | od 5 do 25 g |

**Szukamy:** czterech liczb $(x_1, x_2, x_3, x_4)$ — gramatur każdego składnika — takich, żeby koszt był jak najniższy przy zachowaniu wszystkich wymagań.

## Jak liczymy koszt?

**Wzór na koszt batona**

$$f(x_1, x_2, x_3, x_4) = \frac{\overbrace{0{,}18}^{\text{cena/g}} \cdot x_1 + \overbrace{0{,}06}^{\text{cena/g}} \cdot x_2 + \overbrace{0{,}025}^{\text{cena/g}} \cdot x_3 + \overbrace{0{,}055}^{\text{cena/g}} \cdot x_4}{100} \quad [\text{PLN}]$$

**Czytaj to tak:** pomnóż gramaturę każdego składnika przez jego cenę za gram, dodaj wszystko razem, podziel przez 100 (bo ceny podajemy na 100 g).

**Przykład liczbowy**

Receptura: $x_1 = 25$, $x_2 = 30$, $x_3 = 30$, $x_4 = 15$

$$\begin{aligned} f &= \frac{0{,}18 \cdot 25 \ + \ 0{,}06 \cdot 30 \ + \ 0{,}025 \cdot 30 \ + \ 0{,}055 \cdot 15}{100} \\ &= \frac{4{,}5 + 1{,}8 + 0{,}75 + 0{,}825}{100} \\ &= \frac{7{,}875}{100} \\ &= \mathbf{0{,}6875} \text{ PLN} \end{aligned}$$

# Parametry algorytmu — ustaw raz na początku

Zanim uruchomisz algorytm, wybierasz pięć liczb, które kontrolują jego zachowanie:

| Symbol | Co to jest | Wartość | Intuicja jednym zdaniem |
|---|---|---|---|
| $HMS$ | Rozmiar pamięci | 20 | Tyle receptur pamiętamy jednocześnie |
| $HMCR$ | Szansa na użycie pamięci | 0,90 | 90% razy kopiujesz z pamięci |
| $PAR$ | Szansa na drobną zmianę | 0,30 | 30% razy lekko zmieniasz skopiowaną wartość |
| $BW$ | Jak bardzo zmieniasz | 0,05 | Zmiana to max 5% dopuszczalnego zakresu |
| $NI$ | Ile razy powtarzamy | 10 000 | Liczba prób ulepszenia pamięci |

# Algorytm — 6 kroków

## Krok 1 – Wypełnij pamięć losowymi recepturami

**Co robisz:** generujesz $HMS = 20$ losowych receptur (tutaj uprościmy do 5), każda musi spełniać wszystkie wymagania. Wpisujesz je do tabeli — to jest twoja *pamięć harmonii*.

**Jak wygenerować jedną recepturę ręcznie:**

1. Wylosuj $x_1$ z zakresu [10; 40], np. **25**
2. Wylosuj $x_2$ z zakresu [15; 40], np. **30**
3. Wylosuj $x_4$ z zakresu [5; 25], np. **15**
4. **Oblicz** $x_3 = 100 - x_1 - x_2 - x_4 = 100 - 25 - 30 - 15 = \mathbf{30}$
   (nie losujesz $x_3$ — wyznaczasz go tak, żeby suma była dokładnie 100 g)
5. Sprawdź czy $x_3 \in [15; 35]$ — jeśli nie, zacznij od nowa
6. Sprawdź białko: $0{,}9 \cdot 25 + 0{,}25 \cdot 30 + 0 \cdot 30 + 0{,}14 \cdot 15 = 32{,}1 \geq 20$ ✓
7. Sprawdź tłuszcz: $0{,}5 \cdot 30 + 0{,}06 \cdot 15 = 15{,}9 \leq 20$ ✓

**Po inicjalizacji masz tabelę (pamięć harmonii):**

| Nr | $x_1$ | $x_2$ | $x_3$ | $x_4$ | Koszt [PLN] | Uwaga |
|---|---|---|---|---|---|---|
| $h_1$ | 25 | 30 | 30 | 15 | 0,6875 | |
| $h_2$ | 30 | 20 | 35 | 15 | 0,7300 | |
| $h_3$ | 15 | 35 | 35 | 5 | 0,6725 | ← **najtańsza** |
| $h_4$ | 35 | 15 | 35 | 15 | 0,7975 | ← **najdroższa** |
| $h_5$ | 20 | 25 | 30 | 25 | 0,6750 | |

### Dlaczego tak?

Trzymamy kilka receptur, a nie jedną, bo nie wiemy w którą stronę iść. Różnorodność pamięci = większa szansa na znalezienie lepszego rozwiązania.

## Krok 2 – Wymyśl nową recepturę (improwizacja)

To jest **serce algorytmu**. Dla każdego składnika ($x_1$, $x_2$, $x_4$ osobno) zadajesz sobie dwa pytania po kolei:

### Pytanie 1: Czy korzystam z pamięci?

**Decyzja o pamięci**

Wylosuj liczbę $u \sim U(0,1)$ (równomiernie z przedziału 0 do 1).

Jeśli $u < HMCR = 0{,}9$ $\Rightarrow$ **TAK, kopiuję z pamięci** (90% przypadków)

Jeśli $u \geq HMCR = 0{,}9$ $\Rightarrow$ **NIE, losuję z pełnego zakresu** (10% przypadków)

Jeśli korzystasz z pamięci: wylosuj losowy wiersz $r \in \{h_1, \ldots, h_5\}$ i weź wartość $x_i$ z tego wiersza.

**Dlaczego tak?**

90% czasu budujemy na tym, co już wiemy że działa (receptury z pamięci). 10% czasu eksplorujemy zupełnie nowe wartości — żeby nie utknąć w lokalnym minimum.

### Pytanie 2 (tylko jeśli odpowiedź na pytanie 1 była TAK): Czy lekko zmieniam?

**Decyzja o korekcie**

Wylosuj nową liczbę $u_2 \sim U(0,1)$.

Jeśli $u_2 < PAR = 0{,}3$ $\Rightarrow$ **TAK, dodaję małą zmianę** (30% przypadków)

Jeśli $u_2 \geq PAR = 0{,}3$ $\Rightarrow$ **NIE, zostawiam wartość bez zmian** (70% przypadków)

Jeśli zmieniasz, dodaj losową liczbę $\Delta$:

$$\Delta \sim U\big(-BW \cdot \text{zakres}_i,\ \ +BW \cdot \text{zakres}_i\big)$$

gdzie $\text{zakres}_i = \max_i - \min_i$ (np. dla $x_1$: $40 - 10 = 30$, więc $\Delta \in (-1{,}5;\ +1{,}5)$).

**Dlaczego tak?**

Kopiowanie wprost z pamięci to za mało — algorytm by się "zacementował". Mała korekta pozwala *eksplorować okolice* dobrych receptur. $BW = 5\%$ zakresu to celowo mała wartość: chodzi o lekkie dopasowanie, nie losowy skok.

### Po ustaleniu $x_1$, $x_2$, $x_4$: wylicz $x_3$

Zawsze: $x_3 = 100 - x_1 - x_2 - x_4$. Jeśli wynik jest poza $[15; 35]$, przytnij i skompensuj przez drobną korektę $x_1$ lub $x_2$.

### Przykład liczbowy

Pracujemy na pamięci z Kroku 1. Generujemy nową recepturę krok po kroku.

— **Składnik** $x_1$ —

1. Losuję $u = 0{,}85$. Czy $0{,}85 < 0{,}9$? **TAK** → kopiuję z pamięci.
2. Losuję wiersz: wypadło $h_3$. Biorę $x_1 = 15{,}0$.
3. Losuję $u_2 = 0{,}20$. Czy $0{,}20 < 0{,}3$? **TAK** → stosuję korektę.
4. Zakres $x_1 = 40 - 10 = 30$. Korekta max $= 0{,}05 \cdot 30 = 1{,}5$. Losuję $\Delta = +1{,}2$.
5. $x_1^{\text{new}} = 15{,}0 + 1{,}2 = \mathbf{16{,}2}$

— **Składnik** $x_2$ —

1. Losuję $u = 0{,}93$. Czy $0{,}93 < 0{,}9$? **NIE** → losuję z pełnego zakresu [15; 40].
2. Wylosowana wartość: $x_2^{\text{new}} = \mathbf{28{,}5}$. *(Pomijam pytanie 2 — nie dotyczy.)*

— **Składnik** $x_4$ —

1. Losuję $u = 0{,}76$. Czy $0{,}76 < 0{,}9$? **TAK** → kopiuję z pamięci.
2. Losuję wiersz: wypadło $h_2$. Biorę $x_4 = 15{,}0$.
3. Losuję $u_2 = 0{,}88$. Czy $0{,}88 < 0{,}3$? **NIE** → zostawiam bez zmian.
4. $x_4^{\text{new}} = \mathbf{15{,}0}$

— **Wyznaczam** $x_3$ —

1. $x_3 = 100 - 16{,}2 - 28{,}5 - 15{,}0 = 40{,}3$
2. Ale $40{,}3 > 35$ (max!) → przycinam do $x_3 = 35{,}0$
3. Teraz suma $= 16{,}2 + 28{,}5 + 35{,}0 + 15{,}0 = 94{,}7$. Brakuje 5,3 g.
4. Dokładam brakującą masę do $x_1$: $x_1 = 16{,}2 + 5{,}3 = 21{,}5$ (mieści się w [10; 40]).
5. Sprawdzam sumę: $21{,}5 + 28{,}5 + 35{,}0 + 15{,}0 = 100{,}0$ ✓

**Nowa receptura:** $\mathbf{x}^{\text{new}} = (21{,}5;\ 28{,}5;\ 35{,}0;\ 15{,}0)$, suma $= 100{,}0$ g ✓

### Krok 3 – Policz koszt nowej receptury (z karą za błędy)

**Koszt bazowy** liczymy wzorem z Sekcji 2. Ale jeśli receptura narusza jakieś ograniczenie, **doliczamy karę**.

### Kara za naruszenie ograniczeń

$$F(\mathbf{x}) = \underbrace{f(\mathbf{x})}_{\text{koszt bazowy}} + \underbrace{0{,}5 \cdot \max(0,\ 20 - \text{białko})}_{\text{kara za brak białka}}$$
$$+ \underbrace{0{,}5 \cdot \max(0,\ \text{tłuszcz} - 20)}_{\text{kara za nadmiar tłuszczu}}$$
$$+ \underbrace{0{,}1 \cdot |\text{suma} - 100|}_{\text{kara za złą masę}}$$

**Czytaj każdy człon tak:**

- $\max(0,\ 20 - \text{białko})$ — jeśli białka jest za mało, tu jest niedobór (inaczej zero). Mnożymy przez 0,5 żeby kara była boleśnie duża.
- $\max(0,\ \text{tłuszcz} - 20)$ — analogicznie, nadmiar tłuszczu powyżej 20 g.
- $|\text{suma} - 100|$ — ile gramów "za dużo" lub "za mało" ma baton. Mnożnik 0,1 jest mały, bo po projekcji masa jest bliska 100.

Jeśli wszystko gra — $F(\mathbf{x}) = f(\mathbf{x})$, kara $= 0$.

### Przykład liczbowy

Dla nowej receptury (21,5; 28,5; 35,0; 15,0):

| Co liczę | Obliczenie | Wynik | Status |
|---|---|---|---|
| Koszt bazowy $f$ | $\frac{0{,}18 \cdot 21{,}5 + 0{,}06 \cdot 28{,}5 + 0{,}025 \cdot 35 + 0{,}055 \cdot 15}{100}$ | 0,6600 PLN | – |
| Białko | $0{,}9 \cdot 21{,}5 + 0{,}25 \cdot 28{,}5 + 0 \cdot 35 + 0{,}14 \cdot 15 = 19{,}35 + 7{,}125 + 2{,}1$ | 28,58 g | $\geq 20$ ✓ |
| Tłuszcz | $0{,}5 \cdot 28{,}5 + 0{,}06 \cdot 15$ | 15,15 g | $\leq 20$ ✓ |
| Masa łączna | $21{,}5 + 28{,}5 + 35 + 15$ | 100,0 g | $= 100$ ✓ |
| **Kara** | (wszystkie ograniczenia spełnione) | **0,00** | – |
| $F(\mathbf{x}^{\text{new}})$ | $0{,}6600 + 0{,}00$ | **0,6600 PLN** | – |

## Krok 4 – Wymień najgorszą recepturę w pamięci (jeśli nowa jest lepsza)

### Reguła aktualizacji

1. Znajdź w pamięci recepturę z **najwyższym** kosztem $F$ — nazwijmy ją "najgorsza".
2. Porównaj: czy $F(\mathbf{x}^{\text{new}}) < F(\text{najgorsza})$?
3. Jeśli **TAK**: wykreśl najgorszą, wpisz nową na jej miejsce.
4. Jeśli **NIE**: nic nie rób, pamięć pozostaje bez zmian.

Pamięć może się tylko poprawiać, nigdy pogarszać.

### Przykład liczbowy

Nowa receptura kosztuje $F = 0{,}6600$ PLN.
Najgorsza w pamięci to $h_4$ z kosztem 0,7975 PLN.
$0{,}6600 < 0{,}7975$ $\rightarrow$ **zamieniamy!**

| | $x_1$ | $x_2$ | $x_3$ | $x_4$ | **Koszt** | |
|---|---|---|---|---|---|---|
| $h_1$ | 25 | 30 | 30 | 15 | 0,6875 | |
| $h_2$ | 30 | 20 | 35 | 15 | 0,7300 | |
| $h_3$ | 15 | 35 | 35 | 5 | 0,6725 | ← najtańsza |
| $h_4$ | **21,5** | **28,5** | **35,0** | **15,0** | **0,6600** | **← nowa! (było 0,7975)** |
| $h_5$ | 20 | 25 | 30 | 25 | 0,6750 | |

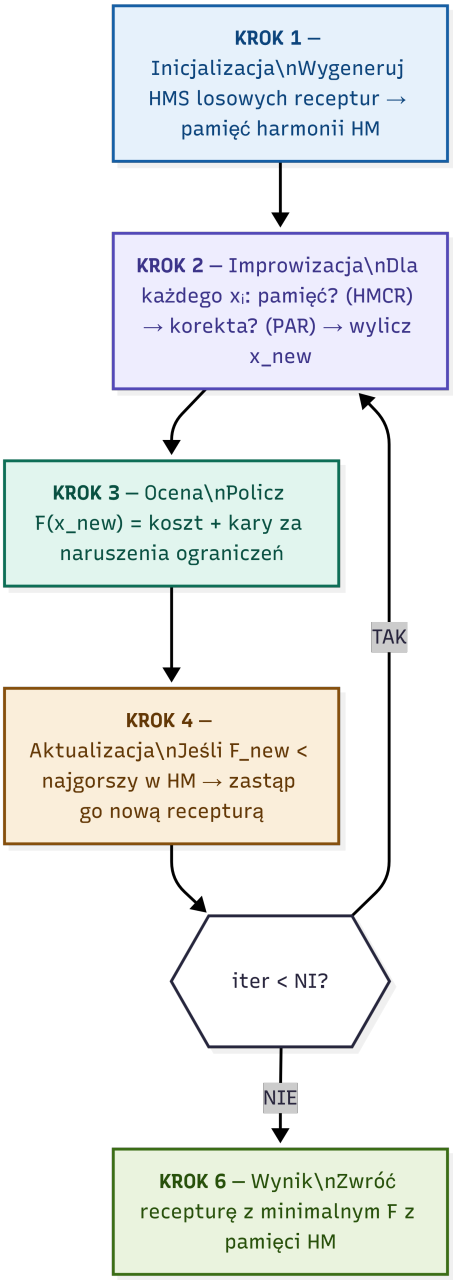
Pamięć jest teraz lepsza niż przed tą iteracją.

### Krok 5 – Powtarzaj kroki 2–4 dokładnie $NI$ razy

Wróć do Kroku 2 i zrób to jeszcze raz. I jeszcze raz. 10 000 razy.
Po każdym powtórzeniu pamięć jest "co najmniej tak dobra" jak przed — nigdy gorsza. Z czasem wszystkie receptury w pamięci zbiegają się ku optimum.

**Co widzisz na wykresie zbieżności:** najtańsza receptura w pamięci kosztuje coraz mniej (lub tyle samo) z każdą iteracją.

### Krok 6 – Odczytaj najlepszą recepturę z pamięci

Po 10 000 iteracjach przejrzyj pamięć i wybierz recepturę z **najniższym** kosztem $F$. To jest twój wynik.
Sprawdź jeszcze raz czy spełnia wszystkie wymagania (w kodzie: pole `feasible`).

# Schemat blokowy — cały algorytm w jednym obrazku